

RECOMMENDATION DOCUMENTATION:

What is a Recommendation System for our project ?

The recommendation system for our project means a system that predicts items based on user preferences, adapting question difficulty for JEE preparation. It offers personalised learning by analysing past performance, ensuring adaptability to individual abilities. Additionally, it dynamically adjusts question difficulty to optimise exam readiness.

General Recommendation System Types :-

1. Popularity Based
2. Content Based .
3. Collaborative Filtering.(User based and Item based)
4. Hybrid Based.

Implementation methodologies used in Existing personalised recommendation systems :

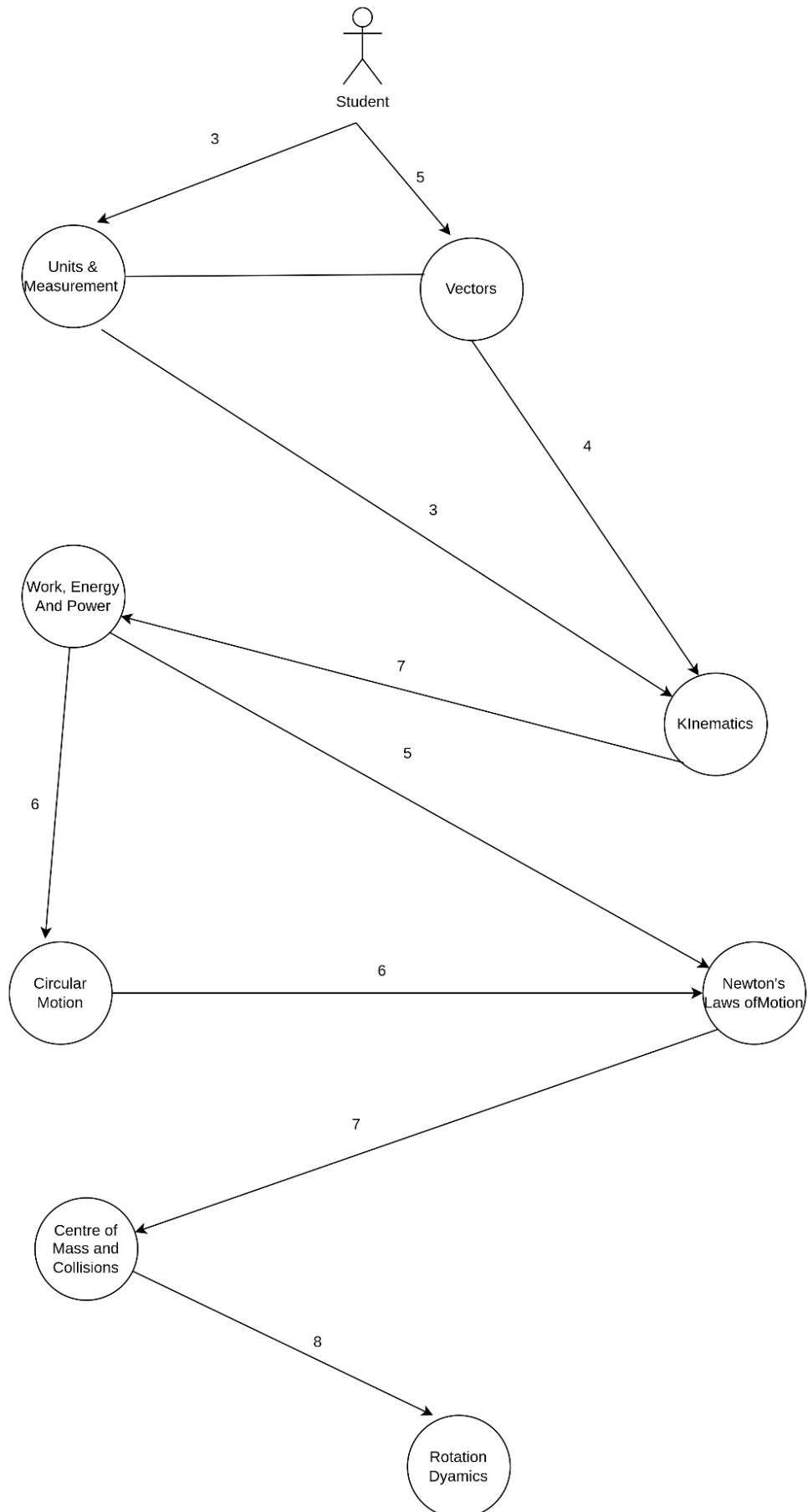
Like the AWS recommendation system.

1. Matrix Factorization
2. Content based Collaborative filtering
3. Item based Collaborative filtering
4. NLP for textual data of reviews.
5. Deep Neural Network for Click through Rate (CTR) prediction.
6. RNN for sequential Recommendation.
7. Real time data processing.
8. A/ B testing and experimentation.

General :

1. Question -> points , levels (easy medium hard)
2. Topics (mean) => questions . => for difficulty.
3. Topic (mode of questions relevance + freq in pyq + time to complete topic and prerequisites) => decided by topic relevance. => edge.
4. User => levels .
5. User => array of topics containing user' level in the topic.

Dependency Graph for Subjects in Physics



Tables Final In Our Project :

- A. Chapter (referring to Section table)
 - a. Chapter_name
 - b. Topic Array
- B. //// Options (This table is referring to Question table) // to be merged in questions table...
 - a. OptionA
 - b. OptionB
 - c. OptionC
 - d. OptionD

C. Questions

Avg_timespent_all_users
IsTestQuestion ***
Sources ***
Acceptance_rate
Test Frequency (test m kitni baar aaya hai)
IsPyq (jee m aaya ya nhi)
Chapter
Subject
Description
Correct answer
Difficulty
Points
Topic
Uploaded By ***
Section
Options ***
Prerequisites_Array # dependency graph.

D. Student

- a. Username
- b. Email
- c. Level
- d. Password
- e. time_spent (amount of time student is devoting to practising questions) => can find average time spent on each question.
- f. Topic array level (dictionary) // proficiency range b/w 0 to 1.
- g. registerDate
- h. exam***
- i. Rating
- j. Ranks array (on every month's 1 date rank will be appended into ranks array)
- k. Level updates Dictionary (previous level - current level : timestamp)
- l. solvedQuestionUniversal Array

```

m. attemptedQuestionUniversal Array
n. incorrectQuestionUniversal Array
o. Physics : {
    solvedQuestions Array
    attemptedQuestions Array
    incorrectQuestions Array
    easy
    Medium
    hard
}
p. Chemistry: {
    solvedQuestions Array
    attempted Questions Array
    incorrect Questions Array
    easy
    Medium
    hard
}
q. Mathematics: {
    solvedQuestions Array
    Attempted Questions Array
    incorrectQuestions Array
    easy
    Medium
    hard
}

```

EXAMPLE:

```

const userInfo = {
  registerDate: "2023-10-12T21:42:22Z",
  rating: 320,
  ranks: [900,743,800,610,620,500,340,350,320],
  level: 4,
  solvedQuestionsUniversal : [1, 2, 3, 4, 5, 6, 7, 8, 9,
10, 11,12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22,23, 24, 25, 26, 27,
28, 29, 30, 31, 32, 33,34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44,45,
46, 47, 48], // submissionId's : Question correct
  incorrectQuestionsUniversal : [1,2,11,12], // incorrect
  attemptedQuestionsUniversal:[13], // attempted ( attempted means
clicked but not submitted anytime)
  levelUpdates:
["2023-10-12T21:42:22Z","2023-11-12T21:42:22Z","2024-01-20T21:42:22Z","
2024-05-12T21:42:22Z"],

```

```

physics: {
    solvedQuestions: [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15], //
submissions Id's : correct Question
    attemptedQuestions: [13], // submission Id's : Attempted but
not submitted Questions ( questions is clicked )
    incorrectQuestions: [1,2,11,12], // submitted but incorrect
    easy: 8,
    medium: 4,
    hard: 3,
},
chemistry: {
    solvedQuestions: [16,17,18,19,20,21,22,23,24,25,26,27],
    attemptedQuestions: [16,17,17,17,17,18,18,19,21],
    incorrectQuestions: [16,20,22,23,24],
    easy: 4,
    medium: 3,
    hard: 4
},
mathematics:{
    solvedQuestions:
[28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,48],
    attemptedQuestions: [28,29,30,31,32,33],
    incorrectQuestions: [28,29,35],
    easy: 10,
    medium: 8,
    hard: 3
},
}

```

E. Subject

- a. Subject_name
- b. Physics sections ['mechanics', 'thermodynamics', 'wave optics', 'nuclear']
- c. Chemistry Sections ['physical', 'inorganic', 'organic']
- d. Mathematics sections ['algebra', 'calculus', 'coordinate geometry', 'differentiation', 'integration']

F. Test =>

- a. Start_date.(Starting date as validity)
- b. End_date.
- c. Type of Test (All Syllabus or one subject ,etc.)
- d. Duration
- e. TotalQuestions
- f. TotalMarks

- G. Test_submission (references test table and student table)
- Score
 - CorrectQuestion
 - WrongQuestion
 - Date
 - TimeTaken
 - Accuracy_Test
- H. Submission (Attempt)
- isRecommendedBefore (flag true or false showing is the question recommended to the corresponding student or not) ***
 - Donestatus (true or false - make entry in submission table
When
User left the question page without making correct attempt then
false
When
User solved the question correctly then
True
)
 - Session_info ***
 - Total Attempts (to derive accuracy)
 - Question_id (references from question table)
 - User_id (references from User table)
 - Submission id (auto generated)
 - timeSpent
 - submissionTimestamp
 - trials : enum[1,2]
 - questionId**
- I. Section (referring to the Subject table)
- Section_name
 - Chapters = ['chapter1', 'chapter2', 'chapter3', 'chapter4']
- J. Topic (referring to chapter table)
- Topic_name
 - Chapter_name

Things to keep in mind : -

Daily Usage of student
Topic/section/chapter/subject time spent
time spent on each question

Adaptation :

1. Change the difficulty of questions after a period.
2. Change the level of students after a session.

How will we do adaptation of questions?

Parameters of Question used in adaptation.

- a. Avg_timespent_all_users :
 - I. ++ $\Rightarrow$ points++
 - II. — $\Rightarrow$ points—
- b. Acceptance_rate.
 - I. ++ $\Rightarrow$ points—
 - II. — $\Rightarrow$ points++
- c. Difficulty
- d. Points (1000 - 10,000) .

Difficulty $\Rightarrow$

1. Easy [1000 - 3000)
2. Medium [3000 - 8000)
3. Hard [8000 - 10,000]

Points $\Rightarrow$ /// points will change after n no of sessions

Where n depends on > 1. No active users.

2. Daily Usage time by each user.

For Now we are taking n = 10 for demonstrative purposes.

Pp $\Rightarrow$ previous points

Pn $\Rightarrow$ New points.

Avp $\Rightarrow$ average time spent by all users. (previous)

Avn $\Rightarrow$ average time spent of all users. (new)

Arp $\Rightarrow$ Acceptance rate. (previous)

Arn $\Rightarrow$ Acceptance rate. (new)

$D1 = Avn - Avp$

$D2 = Arn - Arp$

$Pn = w1 * D1 + w2 * D2 + b \Rightarrow$ Neural n/w or Linear regression....

We will change difficulty based on points.

How will we change the position of students

// Initially every student is provided with a 1000 rating.
// After every session his rating will be updated
And the rating will be updated based on the average rating of the previous session.....

$X = 10$;

Taking 10 levels for now $\Rightarrow$ which will be based on the ratings. (10 levels $\rightarrow$ now a general threshold which we are going to change) $\Rightarrow$ Progress tracking

/**

Maximum Rating $\Rightarrow M$ (of all students)

Minimum Rating $\Rightarrow m$

Level :

// These formulas are for progress tracking.

Difference between two consecutive levels $\Rightarrow d = (M-m)/X$; $X \rightarrow$ No of levels...

Position1 $\Rightarrow [1000 - 1000 + d)$

Position2 $\Rightarrow [1000+d-1000+2d)$

Position 3 $\Rightarrow [1000+2d-1000+3d)$

Position 4 $\Rightarrow [1000+3d-1000+4d)$

Position 5 $\Rightarrow [1000+4d -1000+5d)$

Position 6 $\Rightarrow [1000+5d - 1000 + 6d)$

Position 7 $\Rightarrow [1000+6d-1000+7d)$

Position 8 $\Rightarrow [1000+7d-1000+8d)$

Position 9 $\Rightarrow [1000+8d-1000+9d)$

Position 10 $\Rightarrow [1000+9d -1000+10d)$

**/

Levels used for Recommendation

1. Range $\Rightarrow [1 \text{ to } 10]$.
2. Initially every student is provided with level 1.
3. Level 1 rating $\Rightarrow [1000 - 2000)$
4. Level 2 rating $\Rightarrow [2000-3000)$
5. Level 3 rating $\Rightarrow [3000-4000)$
6. Level 4 rating $\Rightarrow [4000-5000)$
7. Level 5 rating $\Rightarrow [5000-6000)$
8. Level 6 rating $\Rightarrow [6000-7000)$
9. Level 7 rating $\Rightarrow [7000-8000)$
10. Level 8 rating $\Rightarrow [8000-9000)$
11. Level 9 rating $\Rightarrow [9000-10000)$
12. Level 10 rating $\Rightarrow [10000 - \text{INF})$

Note : Rating used for position and level are same.

Algorithm for Recommendation :

1. Topic Recommendation :
 - a. Things using same for all students (From Question Table)
 - i. ExamFreq
 - ii. Question
 - iii. Chapter
 - iv. Subject
 - v. Difficulty
 - vi. Points
 - vii. Topic
 - viii. Section
 - ix. Acceptance_rate
 - x. Test Frequency (test m kitni baar aaya hai)
 - xi. Avg_timespent_all_users

*** Question column will include question option1 option2 option3 option4 subject section chapter topic bs

b. Things used differently for all students (From Student Table) => responsible for personalisation.

- a. Level_Student
- b. rating
- c. time_spent (amount of time student is devoting to practising questions) => can find average time.
- d. Topic array level

// after contests

$$\text{difficulty} = (\text{wRating} * \text{normRating} + \text{wCorrectRate} * (1 - \text{normCorrectRate}) + \text{wAttempts} * \text{normAttempts}) * 100$$

Things we need in our columns : -

1. Exam Frequency
2. Difficulty
3. Points
4. Section
5. Acceptance rate
6. AvgTimeSpent(All users)
7. TimeSpent_Student
8. TopicArrayName
9. TopicArrayLevel
10. Level_Student
11. PredictedQuestionPoints

Points => difficulty

How to prepare Data : -

1. We will prepare an excel file and everyone will prepare data manually.
2. Source of Data : -
 - a. Shikhar Coaching Gwalior.
 - b. Allen Coaching
 - c. And from similar coaching.
3. What we get from it :
 - a. Questions
 - b. Options
 - c. Answer
 - d. Order (rough) > difficulty and roughly points.
4. What we need :
 - a. What question to be predicted after these questions for the corresponding values of students.
 - b. Need avgtimespent , etc parameters for students.
 - c. Which question

1. Badges $\Rightarrow$ 4 $\therefore$
 - a. $\geq 2 \Rightarrow$ Badge1.
 - b. $\geq 5 \Rightarrow$ badge2
 - c. $\geq 7 \Rightarrow$ badge 3
 - d. $\geq 9 \Rightarrow$ Badge4
2. Rank :
3. Level
4. Total question display , subject wise question display
5. Avg time spent per day..
6. Heatmap (with hovering no questions)..... $\Rightarrow$
7. Subject Wise progress.
8. Questions Difficulty change
9. Student level update (initially 1000 rating and level 1)
 - Time Spent
 - Accuracy
 - Question rating solved.

Parameters to take in account

1. Rating of question solved
2. Accuracy for that question
3. Time spent on that question

User prev rating + (points of question * accuracy*k*examFrequency)/timespent;